

Uniform Ordering Optimization for a Youth Soccer League

Prediction to Prescription: Building an AI Agent

Satya Sai Usha Sree Ganteda Daniel Tulloch Affleck Siddharth Shenoy Fabian Alexis
Macias Peñalosa Aditya Ryan Gupta

Team 7 | MGMT 590-037 | Summer 2026

Daniels School of Business – Purdue University

The Decision Problem

Client: CFA (Club Futbol Academy) – youth soccer league, Orange County, CA

The decision: How many uniforms to order for each size and product type – *before* registration demand is known.

Why it is hard:

- 2,000–3,000 orders/season across **14 sizes** × **3 categories**
- **Newsvendor problem** – commit before demand is realized
- **Asymmetric costs** – stockouts up to 5× costlier than overage
- **Jersey lifecycle** – Adidas models discontinued every 2 years. Year 2 models

Cost Asymmetry (Year 2)

Cat.	Over.	Under.	CR
Top	\$15	\$60	0.80
Bottom	\$8	\$25	0.76
Socks	\$1	\$5	0.83

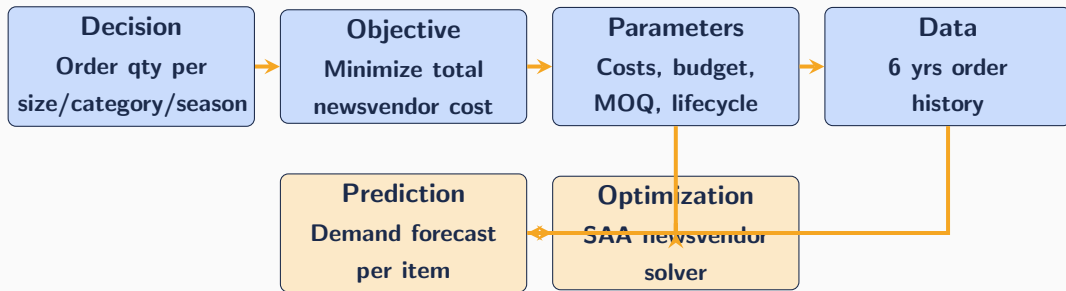
$CR = \frac{c_u}{c_u + c_o} = \text{optimal service level}$

Higher CR $\Rightarrow$ order *above* median demand

Current Process

25 years of “gut feel” – systematic

Backward Mapping: Decision-First Formulation



Objective:

$$\min_{\mathbf{q}} \sum_i [c_i q_i + \frac{1}{S} \sum_s (o_i [q_i - d_{is}]^+ + u_i [d_{is} - q_i]^+)]$$

Constraints:

- Budget: $\sum_i c_i q_i \leq B$ per season
- MOQ: $q_i \geq 6$ per size
- Non-negativity: $q_i \geq 0$

Data: Source and Preparation

Source: CFA Squarespace order export
Proprietary data, authorized by league president

Raw: 90,010 rows $\times$ 52 columns
(2017–2026)

Cleaned: 12,209 line items $\times$ 10 columns

Key columns:

order_date, season, year, product_category,
gender_age, size, quantity, unit_price

Cleaning challenges:

- Size embedded in free-text variant strings
- Product names inconsistent across seasons
- Lifecycle year must be inferred
- Interleaved order-level and line-item rows

Cleaned Data Profile

Metric	Value
Records	12,209
Date range	2020–2026
Categories	3
Size codes	14
Seasons	3 (fall/winter/spring)
Null columns	0

Train / Test Split

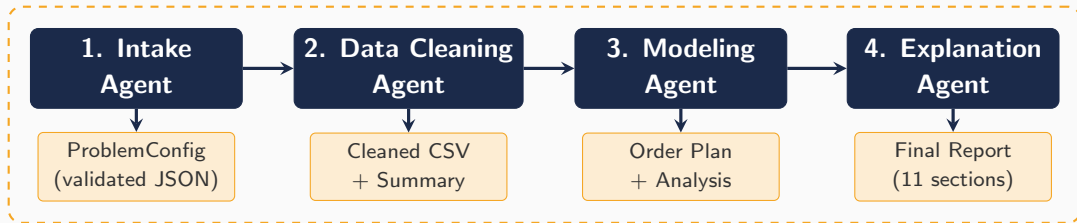
Chronological (no leakage):

Train on historical seasons

Test by backtesting against a held-out period

Agent Architecture: 4-LLM-Agent Pipeline

FastAPI Harness (Orchestrator + Trace Logger)



Design principles:

- LLMs at *judgment* touchpoints only
- Deterministic Python for all numerics
- Sandboxed code execution (agents generate and run Python)
- File-based state passing (auditable, replayable)

Guardrails:

- Pydantic schema validation at every handoff
- Budget constraint enforced structurally
- Agents write structured summaries, not free-form text
- Every step traced in JSON with timing

What Each Agent Does

1. Intake Agent

Input: Problem description (NL or file) + CSV

Tools: Data inspection (peek, sample, describe, unique values), save_config

Output: Validated ProblemConfig JSON

Key: Brainstorms with user, asks clarifying questions, then saves a Pydantic-validated config

2. Data Cleaning Agent

Input: Raw CSV + ProblemConfig

Tools: Data inspection, execute_code (sandbox), save_summary

Output: Cleaned CSV + cleaning summary

3. Modeling Agent

Input: Cleaned CSV + ProblemConfig

Tools: load_data, run_prediction (tunable), run_optimization, validate_results, sensitivity, baseline, execute_code

Output: Order plan + optimizer comparison + summary

Key: Inspects data *before* modeling, fixes granularity issues, adds features, tunes hyperparameters

4. Explanation Agent

Input: All run artifacts

Tools: read_file, list_files, save_report

Output: 11-section final report

Prediction: Methodology

Models compared:

- XGBoost (gradient boosting, tunable depth/LR/trees)
- LightGBM (leaf-wise growth, fast)
- Ridge (linear baseline)
- Each model $\times$ PTO-adjusted variant (6 total)

PTO (Predict-then-Optimize):

Shifts predictions by the newsvendor critical ratio:

$$q^* = \hat{D} \cdot (1 + (CR - 0.5) \cdot \text{strength})$$

Intentionally biases toward the cost-optimal quantile.

Features: Auto-engineered from data

Label-encoded categoricals, year trend, derived features

Why Newsvendor Cost, Not RMSE?

Following Bertsimas–Kallus (2020):

Two models with *identical RMSE* can yield very different ordering costs because the newsvendor penalizes errors **asymmetrically**.

Under-prediction (stockout) costs up to 5 $\times$ more than over-prediction.

$\Rightarrow$ We rank models by the **realized newsvendor cost** of the orders they imply.

Tuning Levers

Formulation: Sample Average Approximation (SAA) of the multi-product newsvendor

$$\min_{\mathbf{q}} \sum_{i=1}^N \left[c_i q_i + \frac{1}{S} \sum_{s=1}^S (o_i(q_i - d_{is})^+ + u_i(d_{is} - q_i)^+) \right]$$

Five solvers benchmarked:

1. **SLSQP** – sequential quadratic programming
2. **L-BFGS-B** – quasi-Newton, box-constrained
3. **Projected Gradient Descent** – first-order, projected
4. **SGD** – stochastic gradient descent
5. **Adam** – adaptive moment estimation

Constraint handling:

- Budget: hard constraint via SLSQP, projection step for first-order methods
- MOQ: floor projection ($q_i \geq 6$)
- Non-negativity: box bounds

What the modeling agent checks:

- All solvers feasible?
- Budget binding or slack?
- Realized cost vs. expected cost

Adaptability: What Can the Instructor Change?

Handled by Re-Run (config change only)

- New constraint
“Add a sustainability cap of 500 total units”
- Changed costs
“Rush shipping now costs 2× more”
- Different budget
“Budget is \$12,000 not \$8,000”
- New season of data
Drop in updated CSV, re-run pipeline
- Shifted objective
“Maximize service level instead”

Needs Re-Engineering

- Fundamentally different problem class
e.g., staff scheduling instead of inventory
- New data source not in existing dataset
- Structurally new constraint types
e.g., per-team allocation, tiered discounts

How it works:

User changes a field in ProblemConfig (via chat or JSON edit) → pipeline re-runs automatically → new order plan in ~10 minutes.

Live Demonstration

1. Upload CSV and problem description
2. Brainstorm with the Intake Agent
3. Approve the ProblemConfig
4. Watch the pipeline run:
Data Cleaning → Modeling → Explanation
5. View the final report with order plan

*We will demonstrate a stakeholder modification –
the instructor may change a constraint or cost parameter mid-demo.*

Results populated from live demo

Prediction

Best model: _____

NV cost: _____

vs. baseline: _____

Optimization

Best solver: _____

Total spend: _____

Budget feasible: _____

Business Impact

Cost savings: _____

vs. current process:

Confidence: _____

The Explanation Agent generates a full 11-section report with order table, sensitivity analysis, and baseline comparison.

Managerial Takeaways

1. **The agent produces a budget-feasible, cost-minimizing order plan** – something the current “gut feel” process cannot guarantee.
2. **Focus attention on youth jerseys** – they drive the majority of spend and carry the most forecasting risk. Adult/women sizes are set by MOQ.
3. **The budget is the binding constraint** – not the forecast model. If the budget is too tight, no algorithm can prevent stockouts.
4. **Cost estimates matter more than demand accuracy** – the plan is highly sensitive to underage cost assumptions and nearly flat to overage.
5. **Re-run the agent when conditions change:** new cost data, updated budget, new season of orders, jersey model transition.
6. **The system is generic** – the same pipeline can optimize any newsvendor-class inventory problem, not just soccer uniforms.

Limitations & Future Work

Current limitations:

- Demand at the size level is inherently noisy – tail spikes are not predictable from order history alone
- Sparse data for rare sizes (AXL, WXL) – MOQ dominates these cells
- Jersey carryover inventory not yet modeled as a constraint
- Pipeline runtime ~ 10 min (dominated by optimizer solver)

Future improvements:

- **Registration/roster data** as a leading demand indicator
- **Hierarchical models** to borrow strength across sparse sizes
- **Carryover-net ordering** as a first-class constraint
- **End-to-end prescriptive model** (Bertsimas–Kallus) training directly on newsvendor loss
- Interactive config editor in the web UI

Adaptable: any change expressible in ProblemConfig fields.

Transparent: every agent writes a structured summary; full trace logged.

Questions?

Thank you!

Satya Ganteda | Daniel Affleck | Siddharth Shenoy | Fabian Macias |

Aditya Gupta

MGMT 590-037 – AI-Enhanced Optimization – Summer 2026